

AIM:

To write a c program to execute the First come First serve scheduling algorithm.

ALGORITHM:

Step 1: Start

Step 2: Initialize variables and arrays for process burst times (a), completion time (b), waiting times (c), and other time variables (t, tt, wt).

Step 3: Input the number of processes n.

Step 4: Input burst times for each process and calculate completion times and waiting times using a loop.

Step 5: Calculate the total turn around time (tt) and total waiting time (wt).

Step 6: Calculate the average waiting time (awt) and average turn around time (att).

Step 7: Display the average waiting time and average turn around time.

Step 8: Display the average gantt chart to visualize the scheduling order.

Step 9: Stop.

EXNO:
02

SHORTEST JOB FIRST
SCHEDULING

DATE:
28.08.23

AIM:

To write a c program to execute the shortest job first scheduling.

ALGORITHM:

- Step 1: Start
- Step 2: Initialize variables and arrays for process burst time (t), process order (P), and other relevant data (wt , awt , avg , s , t , i , j , tt , n , ch)
- Step 3: Input the number of process (n).
- Step 4: Input the burst times for each process and store them in the array $c[]$.
- Step 5: Copy the burst times to another array $a[]$ for reference.
- Step 6: Sort the burst time in ascending order using the array $c[]$, preserving the original order in $a[]$.
- Step 7: Initialize variables i , j , ch .
- Step 8: Perform SJF scheduling and calculate waiting times and turnaround times for each process.
- Step 9: Display a table showing process details (Process number, burst time, waiting time, and turn around time)
- Step 8: Calculate the average waiting time (s) and average turnaround time (t) and display them.
- Step 9: Display a Gantt chart to visualize the scheduling order.
- Step 10: Stop.



AIM:

To write a c program to execute priority scheduling.

ALGORITHM:

Step 1: Start

Step 2: Initialize variables and arrays for process priorities (P), burst times (t), waiting times (wt), turn around times (tt), temporary arrays (b, pro, bi), and average (awt, s, avg, t)

Step 3: Input the number of process (n).

Step 4: For each process input the priority (P[i]) and burst time (t[i]).

Step 5: Create a temporary copies of priority (b, bi), process order (pro), and burst times (tb).

Step 6: Sort the processes based on priority in ascending order while preserving the original order of burst times and other data.

Step 7: Initialize variables i and j for processing.

Step 8: Perform Priority-based scheduling and reorder process order.

Step 9: Initialize variables for calculating waiting times (wt), turn around times (tt), and display tables.

Step 10: Calculate and display waiting times and turn around times for each process.

Step 11: Calculate and display the average waiting time (s) and average turn around time (t)

Step 12: Display Gantt chart to visualize the scheduling order.

Step 13: Stop.



EXNO:
04

ROUND ROBIN SCHEDULING

DATE:
4.9.23

AIM:

To write a c program to execute the Round Robin scheduling.

ALGORITHM:

- Step 1: Start
- Step 2: Initialize variable and arrays for process information.
- Step 3: Input the number of process (n) and the time slice (t_s).
- Step 4: Initialize the number of array and variables to store process information, burst times, waiting times, and other data.
- Step 5: Display the Gantt chart header and initialize necessary variables and counters.
- Step 6: Enter burst times for each process and set up other process-related information.
- Step 7: Perform round-robin scheduling until all processes are completed:

- * Iterate through each process.

- * If the process has remaining burst time greater than the time slice (t_s), update the Gantt chart, adjust time, and process data accordingly.

- * If the process has remaining burst time less than or equal to the time slice, update the Gantt chart, waiting time and process data.

- * Track the number of processes remaining (n).

- Step 8: Display the Gantt chart, including the timeline and process names.

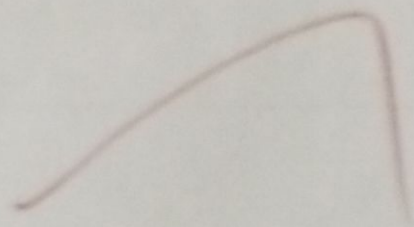
- Step 9: Calculate and display the waiting times and turn around times for each process



--	--	--

Step 10: Calculate and display the average waiting time (aw) and average turn around time (at).

Step 11: Stop.



EXNO:

05

PRODUCER CONSUMER PROBLEM

DATE:

4.9.23

AIM:

To write a C program to execute the Producer consumer problem.

ALGORITHM:

Step 1: Start

Step 2: Initialize variables and arrays of the product, counter, b, ch, buffer[100], in, out.

Step 3: Define function as producer and consumer

Step 4: Input the buffer size (b).

Step 5: Display a menu with options: Product, Consumer, and Exit.

Step 6: Loop until the user chooses to exit (ch is not equal to 3):

- * Display the menu and prompt the user for their choice

- * Based on the choice, either call the producer or consumer function, or exit the program.

Step 7: Implement the producer function:

- * Check if the buffer is full (counter == b). If it is, display "Buffer Full" and return.

- * Prompt the user for the product ID. Store the product in the buffer and update the buffer index (in) and counter.

Step 8: Implement the consumer function:

- * Check if the buffer is empty (counter == 0). If it is, display "Buffer Empty" and return.

- * Consume a product from the buffer, update the buffer index (out), and decrement the counter.
- * Display a message indicating the product consumed.

Step 9: Stop.



ExNO:
06

READER WRITER PROBLEM

DATE:

4.9.23

AIM:

To write a c program to execute the Reader Writer problem.

ALGORITHM:

Step 1: Start

Step 2: Initialize variables and arrays such as $PC[20]$, $SC[20]$, i , no , $flag$, $rd[20]$, $count$.

Step 3: Input the number of process (no).

Step 4: Display a menu to indicate process status:

* 0 - READING

* 1 - WRITING

Step 5: For each process (i from 1 to no):

* Input the process ID ($PC[i]$).

* Input the status ($SC[i]$) indicating if it's reading (0) or writing (1).

Step 6: Display the process IDs and their statuses

Step 7: Create a list of writers by identifying processes with a status not equal to 1 (not writing).

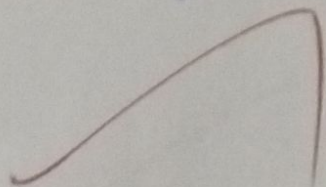
Step 8: For each reading process:

* Display the process ID.

Step 9: If there are reading processes ($flag == 1$), display the list of reading process.

Step 10: If there are no reading processes ($flag == 0$), display "No Reading process".

Step 11: Stop.



EXNO:

07

DINING PHILOSOPHER PROBLEM

DATE:

23.9.23

AIM:

To write a c program to execute the dining philosopher problem.

ALGORITHM:

Step 1: Start

Step 2: Initialize variables and arrays:

* n, f, i, x, y, ch.

* st[25]: Array to store the status of philosophers.

* temp[25]: Array to store temporary flags for philosophers.

Step 3: Show assumptions for the status of philosopher (HUNGRY, EATING, THINKING).

Step 4: Input the status for each philosopher (1, 2 or 3) while validating input.

Step 5: Verify that no two adjacent philosophers are eating at the same time.

Step 6: Display the status of each philosopher are eating at the same time.

Step 7: For each philosopher (i from 1 to n):
* check if they are thinking (status 3) and display "philosopher i is thinking".

* check if they are eating (status 2) and display "philosopher i is eating".

* For philosopher in the HUNGRY state (status 1), check if their neighbors are eating or not.

* Update their status to Eating (2) if the conditions are met and set their temporary flag to 1.

Step 8: Implement the eating function for philosopher.

* check if the neighbors are also in the EATING state and if so, cannot eat.

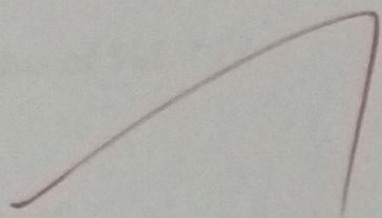


* If a neighbour is currently eating, return and do not eat.

* If the neighbours are not eating and the philosopher's status is not EATING, Set the status to EATING and the temporary flag to 1.

* Handle cases where one or both neighbors are eating and the philosopher cannot eat.

Step 9: Stop.



EXNO:
08

CONTIGUOUS MEMORY ALLOCATION

DATE:

23.9.23

AIM: To write a c program to execute the contiguous memory allocation.

ALGORITHM:

Step 1: Start

Step 2: Initialize variables and data structure:

i, n : counters and variables
struct fh[25]: An array of structures to store block and holes. information (name and size).
Step 3: Implement a function 'fit' to input and initialize the block and holes:

- * Input the total number of blocks and holes (n).

- * Input wheather its a block (1) or a hole (2).

- * Input the size (x) of the block or hole.

Step 4: Implement functions for different allocation strategies (First Fit, Best Fit, worst Fit):

- * Initialize a variable (t) to 0.
- * Iterate through the holes and allocation the hole with the largest size that fits the block size (x).

- * Print the allocation details.

- * 'bFit(x)' for Best Fit:

- create a temporary array (temp) to store available holes sizes.
- Sort and Allocate the hole with the smallest size that fits the block size (x).
- print the allocation details.

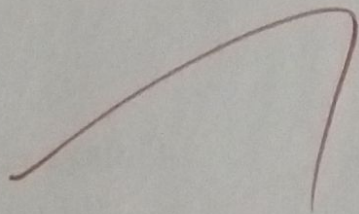
Step 5: Implement a function 'show' to display the allocated blocks and holes.



Step 6: In the 'main' function:

- * Display options for block and hole names.
- * Call the 'fit' function to initialize the memory blocks and holes.
- * Call the 'show' function to display the initial allocation.
- * Input the block size to be inserted (b).
- * Call the allocation functions: 'ffit(b)', 'wfit(b)', and 'bfit(b)' to allocate the block.
- * End the program.

Step 7: Stop.



AIM:

To write a c program to execute the simulation of paging.

ALGORITHM:

- Step 1: Start.
- Step 2: Initialize variable such as phyadr, psi, frame, page, offset, n, pt[20], logadr.
- Step 3: Display a message indicating the simulation of paging.
- Step 4: Input the number of pages (n) and page size (psi).
- Step 5: Input the page table values for each page:
- * Display "Page Number" and prompt for the page's value.
- Step 6: Input the logical address (logadr).
- Step 7: Check if the logical address is valid:
- * If the logical address exceeds the addressable memory range $(n * psi) - 1$, display "Invalid address."
 - * Otherwise, proceed with address translation.
- Step 8: Calculate the offset within the page:
- * $offset = logadr \% psi$.
- Step 9: Calculate the physical address using the page table:
- * $phyadr = (psi * pt[page]) + offset$.
- Step 10: Display the physical address.
- Step 11: Stop.

AIM: To write a c program to execute the Segmentation.

ALGORITHM:

Step 1: Start

Step 2: Initialize variables such as Number of entries (n) in the Segmentation table:
 x, y, sc, pa, ba, ab, li : Integer variables for iteration and address calculations;
 $b[10], l[10], i[10]$: Arrays to store base, limit, and segment number.

Step 3: Display a message indicating the simulation of segmentation.

Step 4: Input the number of entries in the Segmentation table (n).

Step 5: For each entry (x from 0 to $n-1$):

* Input the segment number ($i[x]$).

* Input the limit ($l[x]$).

* Input the base ($b[x]$).

Step 6: Input the segment number to be accessed (sc).

Step 7: Initialize y as -1 to check for invalid segments.

Step 8: Search for the segment in the Segmentation table:

* If a matching segment number ($i[i]$) is found, set y to 1, store the base (ba) and limit (li) for that segment.

* If the segment is not found (y remains -1), it's an invalid segment

Step 9: If the segment is invalid (Y is still -1), display "INVALID SEGMENT" and end the program.

Step 10: Input the byte to be referenced (ab).

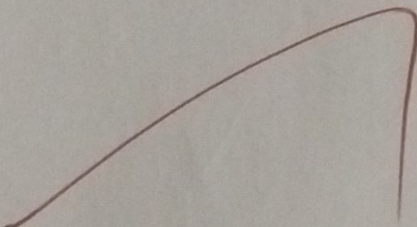
Step 11: check if the referenced byte (ab) is within the segment's limit (li):

* If ab is greater than li , display "Cannot find this Address: Limit only li " where li is the limit for that segment.

Step 12: Calculate the physical address (pa) by adding the base (ba) to the referenced byte (ab)

Step 13: Display the physical address.

Step 14: Stop.



AIM:

To write a c program to execute the FIFO Page Replacement.

ALGORITHM:

Step 1: Start

Step 2: Initialize variables and data structures typically queue or list.

Step 3: Display a message indicating the use of FIFO page Replacement.

Step 4: Input the number of reference string (n) and the page numbers ($a[i]$).

Step 5: Input the number of frames (no).

Step 6: Initialize j to 0, which will be used to cycle through the frames.

Step 7: Display a table with the headers "Ref String" and "Page frame".

Step 8: Iterate through the reference strings (i from 1 to n):

- * Print the current reference string ($a[i]$).

- * Set 'avail' to 0, indicating that the

page is not available in the frames.

Step 9: Check if the current reference string is available in any of the frames:

- * If available, set 'avail' to 1.

Step 10: If the page is not available (avail is 0):

- * Replace the oldest page in the frames with the current reference string ($a[i]$).

- * Update the frames.

- * Increment the page fault count (count).

Step 11: Print the frames after each reference string.

Step 12: Display the total number of page faults (count).

Step 13: Stop.

AIM: To write a c program to execute optimal page replacement.

ALGORITHM:

Step 1: Start

Step 2: Initialize variables and arrays such as i, j, k, n, nf , `Page[max]`, `frame[10]`, `length[10]`, `index`, `highest`, `temp`, `flag`, `Pf`, `found`.

Step 3: Display a message indicating the use of optimal page Replacement.

Step 4: Input the number of frames array (`cnf`) for page replacement.

Step 5: Initialize the page frames array (`frame`) to -1 to represent empty frames.

Step 6: Input the number of reference strings.

Step 7: Display a table with the headers "Ref. String", "Page", and "Frames".

Step 8: Start a loop from $i=0$ to $i=n-1$.

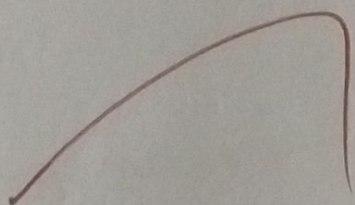
Step 9: Initialize the `flag` to 0, indicating that the page is not yet found in the frames.

Step 10: For each reference string (`Page[i]`), check if it is already in the page frames.

Step 11: If the page is not found in the frame (`flag` is 0), find the best page to replace.

Step 12: Find an empty frame if any (`length[i] == 0`); Set the `index` to the empty frame, and set `found` to 1.

- Step 13: If all frames are occupied (found is 0), find the page with the highest length to the next reference.
- Step 14: Replace the page in the selected frame with the current reference (page[i]).
- Step 15: Print the reference string (page[i]) and the updated page frames (frame).
- Step 16: Increment the page fault count (pf).
- Step 17: Continue the loop for the entire reference string.
- Step 18: Display the total number of page faults (pf).
- Step 19: Stop.



EXNO:
13

LRU PAGE REPLACEMENT

DATE:
14.10.23

AIM: To write a C program to execute the LRU Page replacement.

ALGORITHM:

Step 1: Start

Step 2: Initialize variables and data structure typically queue or list.

Step 3: Display a message indicating the use of LRU Page replacement.

Step 4: Input the number of pages (n).

Step 5: Input the reference string ($P[i]$).

Step 6: Input the number of frames (f).

Step 7: Initialize ' k ' to 0 and add the first page in the reference string to the page frames

' $q[k]$ '.
Step 8: Display the reference string and the initial page frames.

Step 9: Increment ' c ' to count the first page fault.

Step 10: Start a loop from ' $i=1$ ' to ' $i=n-1$ '.

Step 11: For each reference string ($P[i]$), check if it is already in the page frames ' q ':

- * Initialize ' c_1 ' to 0.

- * Loop through the page frames:

- * If ' $P[i]$ ' is not equal to ' $q[j]$ ', increment ' c_1 '.

- * If ' c_1 ' equals the number of frames ' f ',

it means the page is not in the frames.

Step 12: If the page is not in the frames ($c_1 = f$), handle page replacement:

- * Increment the page fault count ' c '.

- * Check if there are empty frames ($k < f$), add the current reference string to the page frames.

- If there are empty frames ($k < f$), add the current reference string to the page frames.



- Update 'k' and print the page frames.
- * If there are no empty frames ($k == f$), calculate which page frame to replace.
 - Calculate the number of page reference since the last use for each frame and store it in 'C2'.
 - Compare the values in 'C2' and find the page frame with the largest count (Least Recently Used).
 - Replace the LRU page frame with the current reference string.
 - Print the updated page frames.
- Step 13: Continue the loop for the entire reference string.
- Step 14: Display the total number of page faults (C).
- Step 15: Stop.



EXNO:
14

CONTIGUOUS FILE ALLOCATION

DATE:
14.10.23

AIM: To write a C program to execute contiguous file allocation.

ALGORITHM:

Step 1: Start.

Step 2: Initialize array disk to represent the disk block and set elements to 0.

Step 3: Create a structure file to store file and give the name and size of the file.

Step 4: Input the number of files (n).

Step 5: For each file (i from 0 to $n-1$), input the following details:

- * Number of blocking occupied by the file ($bc[i]$)
- * Starting block of the file set theme to 1 ($sb[i]$).
- * Initialize $tc[i]$ with $sb[i]$.

Step 6: Display a table with the headers "File name", "start", and "Block Length" to show the file allocation details for each file.

Step 7: Input the file name (x) for which you want to display the details.

Step 8: Display the file name and its length.

Step 9: Display the blocks occupied by the selected file ($cc[x-1][i]$).

Step 10: Stop.

EX NO:
15

LINKED FILE ALLOCATION

DATE:
28.10.23

AIM: To write a C program to execute the Linked File Allocation.

ALGORITHM:

Step 1: Start.

Step 2: Initialize variables and arrays for Freeblock list and Free Allocation table.

Step 3: Display a message indicating the use of contiguous File Allocation.

Step 4: Input the number of file (n).

Step 5: For each file i from 0 to $n-1$, input the following details:

- * Number of blocks occupied by the file ($b[i]$).

- * Starting block of the file ($sb[i]$).

- * Initialize $t[i]$ with $sb[i]$.

- * Allocate contiguous blocks for the file using $c[i][j]$, where j varies from 0 to $b[i]-1$.

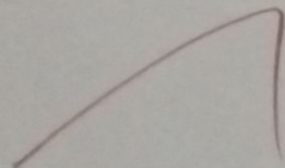
Step 6: Display a table with the headers "File name", "Start", and "Block Length" to show the file allocation details for each file.

Step 7: Input the file name (x) for which you want to display the details.

Step 8: Display the file name and its length.

Step 9: Display the blocks occupied by the selected file ($c[x-1][i]$).

Step 10: Start.



AIM:

To write a C program to execute indexed file allocation.

ALGORITHM:

Start 1: Start

Step 2: Initialize variables and arrays:

 $n, m[20], i, j, sb[20], b[20][20], x.$
 n : Number of files.

 $m[20]$: Array to store the number of blocks occupied by each file.

 $sb[20]$: Array to store the index block for each file.

 $b[20][20]$: 2D array to store block allocation details for each file.

Step 3: Display a message indicating the use of Indexed File Allocation.

Step 4: Input the number of files (n).Step 5: For each file (i from 0 to $n-1$), input the following details.

- * Index block for the file ($sb[i]$).

- * Number of blocks occupied by the file ($m[i]$).

- * Input the block numbers occupied by the file ($b[i][j]$) for each block j from 0 to $m[i]-1$.

Step 6: Display a table with the headers "File", "Index", and "Length" to show the file allocation details for each file.

Step 7: Input the file name (x) for which you want to display the details.

Step 8: Display the file name, its index block, and its length.

Step 9: Display the blocks occupied by the selected file ($b[i][i]$).

Step 10: Stop.



EXNO:

2 17

FIRST COME FIRST SERVE
SCHEDULING

DATE:

30.10.23

AIM: To write a C program to execute First come first serve scheduling.

ALGORITHM:

Step 1: Start.

Step 2: Initialize variables:

h : Starting head position

$a[20]$: Array to store cylinder values.

b : Total seek time

i, j, n : Temporary variables.

Step 3: Display a message indicating the use of First Come First Serve Disk scheduling.

Step 4: Input the number of cylinders (n).

Step 5: Input the starting head head position.

Step 6: Input the cylinder values into the array a .

Step 7: Display the order of visiting the cylinders.

Step 8: Initialize b to 0 (Total seek time).

Step 9: For each cylinder in the array a :

* Calculate the absolute difference between the current head position (h) and the cylinder value ($a[i]$).

* Add this difference to the total seek time (b).

* Update the head position (h) to the current cylinder value ($a[i]$).

Step 10: Display the total seek time (b) in cylinders.

Step 11: Stop.



EXNO:

18

SHORTEST SEEK TIME FIRST
SCHEDULING

DATE:

30.10.23

Aim:

to write a c program to execute shortest seek time first scheduling.

ALGORITHM:

Step 1: Start

Step 2: Initialize variables:

seek : Total seek time.

$t1, t2, n, h, h_1$: Temporary variables.

$a[20]$: Array to store cylinders values.

z : Index for the array.

$d[20]$: Array to store the visited cylinders.

i, j, c : Temporary variables.

Step 3: Display a message indicating the use of the SSTF disk scheduling.

Step 4: Input the number of cylinders (n).

Step 5: Input the cylinder values into the array a .

Step 6: Input the starting head position (h).

Step 7: Initialize variables h_1 to store the original head position and z to 1.

Step 8: Display the order of visiting the cylinders.

Step 9: Initialize seek to 0 (Total seek time).

Step 10: For each cylinder in the array a :

- * call the SSTF function to find the next to visit.
- * calculate and add the seek time to the total seek time.
- * update h_1 to the current head position ($d[i]$).

Step 11: Display the total seek time (seek).

Step 12: Stop.



EXNO:

19

SCAN SCHEDULING

DATE:

6.11.23

AIM:

To write a c program to execute the scan scheduling.

ALGORITHM:

Step 1: Start

Step 2: Initialize variables

Hr : Head position

right left: Variables to calculate seek time for right and left movement

i, j: Loop counters.

h: Temporary head position.

end: Ending position of the disk.

n: Number of cylinders.

a[20]: Array to store cylinder values.

temp, Post: Temporary variables.

Start: Starting position of the disk.

Step 3: Display a message indicating the use of the SCAN Disk Scheduling.

Step 4: Input the ending position of the disk (end).

Step 5: Input the number of cylinders.

Step 6: Input the cylinder values into the array a.

Step 7: Sort the cylinder values in ascending order

Step 8: Input the starting head position (h).



Step 9: Initialize variables h_r to store the original head position, $right$ and $left$ to 0.

Step 10: Find the position in the stored array where the head is less than the cylinder value (Post).

Step 11: Display the order of visiting the cylinders in the right direction.

Step 12: Calculate seek time (right) for moving to the right side:

- * Visit the cylinders from post to n .
- * Calculate and add the seek time for each movement.
- * Update h_r to the current head position $a[i]$.

Step 13: Display the total seek time (right) for moving to the right side.

Step 14: Display the order of visiting the cylinders in the left direction.

Step 15: Calculate seek time (left) for moving to the left side:

- * Visit the cylinders from post - 1 to 1.
- * Calculate and add the seek time for each movement.
- * Update h_r to the current head position $a[i]$.

Step 16: Display the total seek time (left) for moving to the left side.

Step 17: Stop.



EXNO :

20

C-SCAN SCHEDULING

DATE:

6.11.23

AIM:

To write a C program to execute the C-scan scheduling.

ALGORITHM:

Step 1: Start

Step 2: Initialize 'left' and 'right' to 0.

Step 3: Initialize start, 'i', 'j', 'temp' and 'post'.

Step 4: Prompt for disk parameters: 'end', 'n', 'a[]' and 'h'.

Step 5: Sort 'a[]' in ascending order.

Step 6: Find the 'post' where 'h' should start moving right.

Step 7: Visit cylinders from 'post' to end, updating 'right' and 'h'.

Step 8: calculate seek time to move from last cylinder to 'end'.

Step 9: Visit cylinders from start to 'post-1', updating 'right' and 'h'.

Step 10: calculate seek time to move from 'post-1' to start.

Step 11: Display total seek time 'right'.

Step 12: Stop.

